

Eficiência Computacional

“ Um algoritmo pode ser analisado por sua reusabilidade, legibilidade, corretude, e **eficiência**. ”

Análise de Eficiência

Busca avaliar se um algoritmo resolve um problema da melhor maneira.

Análise de Eficiência

Pode-se avaliar o consumo de recursos
(memória, disco, rede) ou **tempo de
resposta.**

Análise de Eficiência

Para analisar o desempenho de algoritmos com relação ao **tempo de resposta**, podemos tomar duas abordagens diferentes: **Análise Empírica** ou **Análise de Complexidade**.

Análise Empírica

Consiste em medir o tempo de execução de um algoritmo.

Análise Empírica

Pode ser influenciada por diferentes configurações de computadores. **Ex.:** velocidade da **CPU**, velocidade de **memória**, quantidade de programas executando em simultâneo (**concorrência**), etc.

Análise de Complexidade

É a análise científica (formal) de um algoritmo, que visa modelar matematicamente o tempo de resposta de um algoritmo em relação ao tamanho de sua entrada.

Tempo x Entrada

Existe uma relação direta entre o tempo de resposta e o tamanho da entrada de um algoritmo.

Tempo x Entrada

Muitas vezes apenas questões de hardware são levadas em consideração (disco, memória, rede). Porém existe um fator mais importante: **a complexidade do algoritmo.**

Intuição

“ Quanto maior o tamanho da entrada, maior o tempo de resposta. ”

Intuição Errada

“ Se uma entrada de 10 elementos executa
50 operações, uma entrada de 20
elementos executa 100 operações ”

“ A relação entre tamanho de entrada e tempo de execução depende de **como** o algoritmo foi construído e precisa ser analisada caso a caso. ”

Análise de Complexidade

Visa definir uma relação matemática entre o tamanho da entrada n e o tempo de execução T .

Tempo de Reposta

O tempo de resposta T deve ser interpretado como **quantidade de operações**.

Contagem de Custo

Para cotar quantas operações um algoritmo realiza, precisaremos adotar certas diretivas (regras).

Diretivas

1. Comandos de atribuição, entrada, saída e testes tem custo 1. O comando senão, não tem custo.

Diretivas

2. Blocos `if` ou loop, podem serem executados um n° de vezes variável no mesmo algoritmo. Nesse caso precisam ser analisadas as 3 possibilidades: pior caso, melhor caso, caso médio. Com preferência pelo pior caso.

Diretivas

3. O n° de operações de um loop pode ser representado por meio de um somatório. Blocos aninhados implicam somatórios de somatórios. Resolveremos do mais interno para o mais externo.

Exemplo de Análise

```
# imagine uma lista "l1"  
soma = 0  
for item in l1:  
    soma += item
```

Exemplo de Análise

```
# imagine uma matriz quadrada "mat"  
soma = 0  
for i in range(n):  
    for j in range(n):  
        soma += mat[i][j]
```

Análise Assintótica

Buscaremos avaliar como o algoritmo se comporta ao variar n dentro do intervalo $[-\infty, +\infty]$.

Comportamento Assintótico

Ao variar n dentro de $[-\infty, +\infty]$, obteremos o que é denominado como **comportamento assintótico**.

Classes Assintóticas

Podemos agrupar (classificar) os algoritmos em diversas classes, de acordo com o seu tipo de comportamento assintótico.

Classes Assintóticas

1. Constante: $O(1)$
2. Logarítmica: $O(\lg n)$
3. Linear: $O(n)$
4. Logaritmo-Linear: $O(n \lg n)$

Classes Assintóticas

- 5. Quadrática: $O(n^2)$
- 6. Cúbica: $O(n^3)$
- 7. Exponencial: $O(c^n)$, sendo c uma constante qualquer.

Exemplo

Este algoritmo pertence à classe linear:
 $O(n)$.

```
# imagine uma lista "l1"  
soma = 0  
for item in l1:  
    soma += item
```

Exemplo

Este algoritmo pertence à classe quadrática: $O(n^2)$.

```
# imagine uma matriz quadrada "mat"
soma = 0
for i in range(n):
    for j in range(n):
        soma += mat[i][j]
```

Exemplo Gráfico (Geogebra)

Problemas Polinomiais

São problemas que podem ser expressos por uma função polinomial.

$$f(n) = n^1 + n^2 + n^3$$

Problemas não Polinomiais

A única classe não polinomial são os algoritmos exponenciais.

$$f(n) = c^n$$

“ Os algoritmos não polinomiais **não** podem ser executados em tempo computacional razoável para entradas relativamente grandes. ”

Tratabilidade

“ Sempre que possível, iremos preferir as soluções polinomiais, pois nos darão uma resposta em tempo computacionalmente **tratável**. ”

Tratabilidade

“ Um problema computacionalmente **intratável** é um que não possua resolução em tempo polinomial. ”